

PDSP 2025, Lecture 03, 14 August 2025

Generating sequences of numbers

- `range(n)` generates the sequence
- Use `list(range(n))` to display as

```
n = 17
```

```
range(n) # Like a list, but not
range(0, 17)
```

```
list(range(n)) # Make it into a list
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
```

- `range(n)` translates to `range(0, n)`, implicitly starting with `0`
- Can add an explicit starting point: `range(i, n)` generates `i, i+1, ..., n-1`

```
list(range(2, n))
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
```

- If the starting point is $\geq$ the target, `range` generates an empty sequence

```
list(range(3, 3))
[]
```

```
list(range(7, 4))
[]
```

Numbers in Python

- Numbers in Python can be integers (`int`) or reals -- actually rationals -- (`float`)
- Internal representation is different, but arithmetic operation symbols are *overloaded* to apply to both types of numbers
- `+`, `-`, `*` stand for addition, subtraction, multiplication, as usual
- `/` is division, and always produces a `float`

```
8/4
2.0
```

- There are separate operators for *quotient* (`//`) and *remainder* (`%`)
 - These can also be applied to `float` arguments, but the answer is also `float`

```
8//4
2
```

```
7 % 3
1
```

```
8.0//3.0, 8.0 % 5.0
(2.0, 3.0)
```

```
9.3//3.05, 9.3 % 3.05
(3.0, 0.15000000000000124)
```

Frequently asked questions

View release notes

Search code snippets ⌘/Ctrl+Alt+P

Report a bug

Report Drive abuse

Send feedback

View Terms of Service

View in English

Data types

- A data type is a set of values with associated operations
- Python has two numeric data types, `int` and `float`
- In the IPL example, we saw text data, which is of type `String` -- we shall examine this later
- The boolean data type has two values `True` and `False`

✓ Checking if a number is prime

- Checking if `n` is a prime: assume it is, and flag that is not if we find a factor between `2` and `n-1`

```
n = 17
isprime = True
for i in range(2,n):
    if n % i == 0:
        isprime = False
```

```
n, isprime
```

```
(17, True)
```

```
n = 18
isprime = True
for i in range(2,n):
    if n % i == 0:
        isprime = False
```

```
n, isprime
```

```
(18, False)
```

✓ Optimising the search for factors

- Factors occur in pairs, sufficient to check from 2 to $\sqrt{n}$
- Python has a function `sqrt` to compute square roots
- However it is not automatically available

```
sqrt(n)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 sqrt(n)

NameError: name 'sqrt' is not defined
```

✓ Libraries

- Libraries are collections of code implementing different groups of functions relevant to a given theme
- We will later see libraries specific to data science, machine learning
- The `math` library has mathematical functions like `sqrt`, `log`, `sin`, `cos` etc
- We `import` the `math` library to use it
 - Note that we use `math.sqrt` to tell Python the full context of the function `sqrt`
 - This is useful in case two different libraries have different functions with the same name

```
import math
```

```
n = 17
math.sqrt(n)
```

```
4.123105625617661
```

✓ Optimised primality checking

- We can optimize our search for factors by restricting the range to `(2,math.sqrt(n))`
- `range` expects only `int` arguments, so use `int()` to convert `math.sqrt(n)` to an `int` -- truncates the fractional part

```
n = 17
isprime = True
for i in range(2,math.sqrt(n)):
    if n % i == 0:
        isprime = False
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[19], line 3
      1 n = 17
      2 isprime = True
----> 3 for i in range(2,math.sqrt(n)):
      4     if n % i == 0:
      5         isprime = False

TypeError: 'float' object cannot be interpreted as an integer
```

```
n = 17
isprime = True
for i in range(2,int(math.sqrt(n))): # int(...) truncates a float to an int
    if n % i == 0:
        isprime = False
```

```
isprime
```

```
True
```

- We have to be careful, because `range(j,m)` stops at `m-1`
- The code above wrongly claims `25` is a prime -- the search for factors runs from `2` to `4` rather than `2` to `5`

```
n = 25
isprime = True
for i in range(2,int(math.sqrt(n))): # int(...) truncates a float to an int
    if n % i == 0:
        isprime = False
```

```
isprime
```

```
True
```

- To fix this, modify the upper bound of `range` to `sqrt(n)+1`

```
n = 25
isprime = True
for i in range(2,int(math.sqrt(n))+1): # int(...) truncates a float to an int
    if n % i == 0:
        isprime = False
```

```
isprime
```

```
False
```

✓ Large and small numbers

- Python allows us to work with very large (and very small numbers)
- The operator `**` is exponentiation

```
7**3, 2**12
```

```
(343, 4096)
```

- What is $2^{2^{12}}$, in other words, 2^{4096} ?

```
2**(2**12)
```

```
10443888814131525066917527107166243825799642490473837803842334832839539079715574568488268119349975583408901067144
```

```
2**24
```

```
16777216
```

```
2** (2**24)
```

```
-----
ValueError                                Traceback (most recent call last)
File ~/python-venv/lib/python3.13/site-packages/IPython/core/formatters.py:770, in
PlainTextFormatter.__call__(self, obj)
    763 stream = StringIO()
    764 printer = pretty.RepresentationPrinter(stream, self.verbose,
    765     self.max_width, self.newline,
    766     max_seq_length=self.max_seq_length,
    767     singleton_pprinters=self.singleton_pprinters,
    768     type_pprinters=self.type_pprinters,
    769     deferred_pprinters=self.deferred_pprinters)
--> 770 printer.pretty(obj)
    771 printer.flush()
    772 return stream.getvalue()

File ~/python-venv/lib/python3.13/site-packages/IPython/lib/pretty.py:386, in RepresentationPrinter.pretty(self,
obj)
    383 for cls in _get_mro(obj_class):
    384     if cls in self.type_pprinters:
    385         # printer registered in self.type_pprinters
--> 386         return self.type_pprinters[cls](obj, self, cycle)
    387     else:
    388         # deferred printer
    389         printer = self._in_deferred_types[cls]

File ~/python-venv/lib/python3.13/site-packages/IPython/lib/pretty.py:786, in _repr_pprint(obj, p, cycle)
    784 """A pprint that just redirects to the normal repr function."""
    785 # Find newlines and replace them with p.break_()
--> 786 output = repr(obj)
    787 lines = output.splitlines()
    788 with p.group():

ValueError: Exceeds the limit (4300 digits) for integer string conversion; use sys.set_int_max_str_digits() to
increase the limit
```

- How about 2^{-4096} ?

```
2**(-(2**12))
```

```
0.0
```

- The value has become too small to distinguish from zero
- On the other hand 2^{-1024} works

```
2**(-(2**10))
```

```
5.562684646268003e-309
```